

**LAPORAN TUGAS BESAR PENERAPAN ALGORITMA
PADA APLIKASI PASARKITA**

MATA KULIAH STRATEGI ALGORITMA



DISUSUN OLEH:

RADITYA RIZKI RAHARJA	714240041
RIDWAN HAKIM RAMADHAN	714240050
MUHAMMAD RASHID AL SAVERO	714240006

DOSEN PENGAMPU:

RONI HABIBI, S.KOM., M.T., SFPC

**PROGRAM STUDI DIV TEKNIK INFORMATIKA
UNIVERSITAS LOGISTIK & BISNIS INTERNASIONAL
BANDUNG**

2026

DAFTAR ISI

DAFTAR ISI	i
DAFTAR TABEL	ii
DAFTAR PSEUDOCODE	iii
RINGKASAN	iv
BAB 1 PENDAHULUAN ANALISIS	1
1.1 Konteks Aplikasi	1
1.2 Fokus Analisis	1
1.3 Fitur yang Dianalisis	1
1.4 Sumber Data Analisis	2
BAB 2 GAMBARAN FITUR DAN REFERENSI	3
2.1 Analisis Jurnal sebagai Dasar Pemilihan Algoritma	3
2.2 Alasan Khusus Aplikasi Menggunakan Algoritma Ini	4
2.3 Alur Fitur yang Dianalisis	4
2.4 Ringkasan Input, Proses, dan Output	5
BAB 3 ANALISIS ALGORITMA DAN OUTPUT	6
3.1 Analisis Greedy pada Rekomendasi Produk	6
3.2 Analisis Greedy pada Sorting Produk	7
3.3 Analisis Greedy pada Fee Marketplace	8
3.4 Analisis KMP pada Pencarian Produk	9
3.5 Analisis String Matching pada Pencarian Pesanan	10
3.6 Kesimpulan Analisis Output	11
DAFTAR PUSTAKA	13

DAFTAR TABEL

1	Fitur PasarKita yang Dianalisis	2
2	Analisis Jurnal dan Hubungannya dengan PasarKita	3
3	Keputusan Algoritma pada Fitur PasarKita	4
4	Ringkasan Input, Proses, dan Output Algoritma	5
5	Output dari Penerapan Algoritma	12

DAFTAR PSEUDOCODE

1	Perhitungan fee marketplace PasarKita	9
2	Kompleksitas Knuth-Morris-Pratt	10

RINGKASAN

Laporan ini menyajikan analisis penerapan algoritma pada aplikasi marketplace PasarKita. Fokusnya bukan membangun pembahasan akademik yang panjang, melainkan menunjukkan bagian aplikasi yang memakai algoritma, alasan algoritma tersebut dipilih, potongan kode atau pseudocode yang relevan, serta contoh output yang dihasilkan. Dua algoritma utama yang dianalisis adalah Greedy dan String Matching berbasis Knuth-Morris-Pratt. Greedy digunakan pada rekomendasi produk, sorting produk berdasarkan penjualan atau rating, dan perhitungan fee marketplace 2%. String Matching/KMP digunakan untuk pencarian produk dan pencarian data pesanan. Referensi tetap digunakan sebagai penguat analisis, misalnya teori adopsi teknologi [1], sistem rekomendasi [2], Greedy pada konteks e-commerce [3], dan penerapan KMP pada e-katalog [4]. Hasil analisis menunjukkan bahwa algoritma sederhana dapat memberi dampak langsung pada fitur marketplace: rekomendasi menjadi lebih relevan, produk dapat diurutkan berdasarkan skor, fee dapat dihitung cepat, dan pencarian produk/pesanan menjadi lebih mudah digunakan.

BAB 1

PENDAHULUAN ANALISIS

1.1 Konteks Aplikasi

PasarKita adalah aplikasi marketplace digital untuk produk UMKM. Aplikasi ini mempertemukan tiga aktor utama: pembeli, penjual, dan admin. Pembeli menggunakan aplikasi untuk melihat katalog, mencari produk, memasukkan produk ke keranjang, melakukan checkout, dan melihat status pesanan. Penjual menggunakan aplikasi untuk mengelola produk, stok, pesanan, serta dashboard toko. Admin menggunakan aplikasi untuk memantau user, produk, order, analytics, dan kondisi integrasi.

Marketplace seperti PasarKita membutuhkan algoritma karena data yang ditampilkan tidak cukup hanya disimpan dan diambil apa adanya. Produk perlu direkomendasikan, diurutkan, dicari, dan dihitung biayanya. Recommender system membantu pengguna memilih item ketika pilihan terlalu banyak [2]. Dari sisi organisasi, penggunaan teknologi informasi juga relevan untuk membantu proses adopsi digital pada level usaha atau organisasi [1]. Karena itu, PasarKita cocok dijadikan objek analisis penerapan algoritma dasar.

1.2 Fokus Analisis

Laporan ini hanya membahas fitur yang sudah memiliki penerapan algoritma jelas di dokumentasi dan kode aplikasi. Fokusnya adalah sebagai berikut:

- Algoritma Greedy pada rekomendasi produk, sorting produk, dan fee marketplace.
- String Matching/KMP pada pencarian produk buyer dan seller.
- String Matching multi-field pada pencarian pesanan.
- Potongan kode, pseudocode, dan contoh output dari penerapan algoritma.

Laporan ini tidak mengklaim bahwa PasarKita sudah memakai machine learning recommendation. Referensi Epsilon Greedy digunakan sebagai catatan pengembangan, bukan sebagai implementasi saat ini [5].

1.3 Fitur yang Dianalisis

Fitur yang dianalisis dipilih karena langsung berhubungan dengan kebutuhan marketplace.

Tabel 1: Fitur PasarKita yang Dianalisis

Fitur	Masalah yang Diselesaikan	Algoritma
Rekomendasi produk	Pembeli butuh produk yang relevan dari banyak pilihan	Greedy scoring
Sorting produk	Produk perlu ditampilkan berdasarkan performa penjualan atau rating	Greedy ranking
Fee marketplace	Total checkout perlu dihitung cepat dan konsisten	Greedy satu langkah
Pencarian produk	Buyer dan seller perlu menemukan produk berdasarkan keyword	KMP/String Matching
Pencarian pesanan	Seller perlu mencari order berdasarkan beberapa field	String Matching multi-field

1.4 Sumber Data Analisis

Analisis dilakukan dari tiga sumber utama. Pertama, dokumentasi algoritma pada folder `docs/algorithms`. Kedua, source code aplikasi yang berkaitan dengan rekomendasi, fee, pencarian produk, dan pencarian pesanan. File utama yang ditinjau adalah modul rekomendasi frontend, utilitas fee, utilitas KMP, service produk, dan service order. Ketiga, referensi yang sudah dikumpulkan dalam file BibTeX `src/MyCollection.bib`.

BAB 2

GAMBARAN FITUR DAN REFERENSI

2.1 Analisis Jurnal sebagai Dasar Pemilihan Algoritma

Referensi pada bagian ini tidak hanya dipakai sebagai kutipan teori, tetapi sebagai alasan mengapa algoritma tertentu masuk akal untuk PasarKita. Cara membacanya sederhana: jurnal memberi konteks masalah, lalu konteks itu diterjemahkan menjadi keputusan fitur di aplikasi.

Tabel 2: Analisis Jurnal dan Hubungannya dengan PasarKita

Referensi	Isi penting jurnal	Hubungan dengan Aplikasi
Oliveira dkk. [1]	Membahas model adopsi teknologi pada level organisasi, termasuk kerangka DOI dan TOE. Intinya, teknologi dipakai ketika bisa membantu organisasi bekerja lebih efisien dan menyesuaikan diri dengan lingkungan bisnis.	PasarKita dibuat sebagai kanal digital untuk UMKM. Karena itu, fitur marketplace tidak cukup hanya menampilkan data; aplikasi perlu membantu proses jual-beli berjalan lebih cepat, mudah dicari, dan mudah dipantau.
Resnick dkk. [2]	Menjelaskan bahwa recommender system membantu user memilih item ketika pilihan terlalu banyak. Masalah utamanya bukan kekurangan data, tetapi terlalu banyak alternatif yang harus dipilih user.	Produk UMKM dalam marketplace bisa bertambah banyak. PasarKita memakai rekomendasi berbasis skor agar buyer tidak harus menelusuri semua produk satu per satu.
Tanujaya & Musyaffa [3]	Membandingkan Greedy dan Dynamic Programming pada optimasi diskon keranjang e-commerce. Greedy ditunjukkan sebagai pendekatan yang praktis dan cepat, walaupun tidak selalu menjamin hasil optimal global seperti pendekatan yang lebih berat.	Fitur rekomendasi, sorting, dan fee PasarKita membutuhkan respons cepat. Karena targetnya ranking dan perhitungan sederhana, Greedy lebih cocok daripada algoritma optimasi yang terlalu kompleks.
Marbun dkk. [4]	Menerapkan Knuth-Morris-Pratt pada e-katalog perpustakaan. Fokusnya adalah mencocokkan kata kunci dengan data katalog agar pencarian item lebih efisien.	PasarKita memiliki kasus yang mirip dengan e-katalog: user mengetik keyword, lalu sistem mencari produk yang namanya mengandung keyword tersebut. Karena itu, KMP sesuai untuk pencarian produk.
Al-howaide dkk. [6]	Membahas algoritma string matching dan perbandingan cara pencocokan string. Poin pentingnya adalah setiap algoritma pencarian memiliki cara mengurangi perbandingan karakter yang tidak perlu.	Pencarian produk dan pesanan tidak boleh terasa lambat ketika data bertambah. String Matching dipakai agar keyword bisa dicocokkan ke nama produk, ID transaksi, atau nomor tracking secara terarah.
Crochemore & Hancart [7]	Menjelaskan pattern matching sebagai masalah dasar dalam pemrosesan string: mencari kemunculan pattern di dalam teks.	Fitur pencarian PasarKita pada dasarnya adalah pattern matching. Keyword adalah pattern, sedangkan nama produk/order field adalah teks yang dicari.

2.2 Alasan Khusus Aplikasi Menggunakan Algoritma Ini

Tabel berikut bisa dipakai sebagai ringkasan utama saat presentasi. Isinya langsung menghubungkan masalah aplikasi, alasan pemilihan algoritma, dan output yang terlihat.

Tabel 3: Keputusan Algoritma pada Fitur PasarKita

Algoritma	Masalah di aplikasi	Mengapa cocok	Penerapan pada Aplikasi
Greedy scoring	Buyer butuh rekomendasi produk yang relevan tanpa proses berat.	Mengambil produk dengan skor terbaik berdasarkan kategori yang sering dilihat, jumlah terjual, dan rating. Cocok untuk rekomendasi awal karena cepat dan mudah dijelaskan.	Daftar produk rekomendasi: stok tersedia, belum dilihat, dan kategori/skornya relevan.
Greedy ranking	Produk perlu diurutkan berdasarkan terlaris atau rating tertinggi.	Setiap produk punya skor lokal. Sistem cukup mengurutkan skor tersebut tanpa mencari kombinasi global.	Tabel produk terurut dari sold units atau rating terbesar.
Greedy satu langkah	Checkout perlu menghitung fee marketplace dengan konsisten.	Fee selalu 2% dari subtotal, sehingga cukup dihitung langsung dalam satu langkah.	Subtotal, fee marketplace, dan total pembayaran.
KMP	Buyer dan seller perlu mencari produk berdasarkan keyword.	KMP memakai prefix table sehingga ketika terjadi mismatch, pencarian tidak selalu diulang dari awal. Ini cocok untuk exact substring search pada nama produk.	Keyword sep menghasilkan produk seperti Sepatu Running Lokal.
String Matching multi-field	Seller perlu mencari order melalui order ID, nama produk, transaction ID, atau tracking ID.	Masalahnya bukan hanya mencari satu nama produk, tetapi mencocokkan keyword ke beberapa field. Pendekatan string matching membuat pencarian lebih fleksibel.	Keyword transaksi atau tracking menampilkan order yang cocok.

Dari tabel tersebut, Greedy dipilih ketika PasarKita perlu mengambil keputusan cepat berdasarkan skor lokal. KMP dan String Matching dipilih ketika fitur membutuhkan pencocokan keyword pada teks. Referensi Umami & Rahmawati [5] tidak dipakai untuk mengklaim implementasi saat ini, tetapi menjadi catatan pengembangan jika rekomendasi PasarKita ingin dibuat lebih adaptif dengan pola eksplorasi dan eksploitasi.

2.3 Alur Fitur yang Dianalisis

Secara umum, fitur yang dianalisis mengikuti alur input, proses algoritma, dan output. Rekomendasi produk menerima input berupa daftar produk dan histori produk

yang pernah dilihat. Sistem menghitung skor setiap produk, mengurutkan produk berdasarkan skor, lalu mengambil produk teratas. Sorting produk bekerja dengan pola serupa, tetapi skor utamanya berasal dari unit terjual atau rating. Fee marketplace menerima subtotal dan langsung menghasilkan fee serta total pembayaran.

Pada pencarian, input utama adalah keyword. Keyword tersebut dicocokkan dengan nama produk atau field order. Untuk pencarian produk, PasarKita menggunakan KMP yang bersifat case-insensitive. Untuk pencarian pesanan, keyword dapat dicari pada order ID, nama produk, transaction ID, dan tracking ID.

2.4 Ringkasan Input, Proses, dan Output

Tabel 4: Ringkasan Input, Proses, dan Output Algoritma

Fitur	Input	Proses	Output
Rekomendasi produk	Produk dan histori lihat	Hitung skor lokal, sort, ambil top-N	Produk rekomendasi
Sorting produk	Data produk, sold units, rating	Ranking berdasarkan skor utama	Produk terurut
Fee marketplace	Subtotal checkout	Hitung 2% dari subtotal	Fee dan total
Pencarian produk	Keyword dan nama produk	KMP case-insensitive	Produk yang cocok
Pencarian pesanan	Keyword dan field order	Matching beberapa field	Order yang cocok

BAB 3

ANALISIS ALGORITMA DAN OUTPUT

3.1 Analisis Greedy pada Rekomendasi Produk

Greedy pada rekomendasi produk digunakan untuk memilih produk terbaik berdasarkan skor lokal. Sistem tidak mencoba semua kombinasi produk, tetapi langsung memberi skor pada setiap kandidat, mengurutkannya, lalu mengambil produk teratas. Pendekatan seperti ini cocok untuk fitur interaktif karena lebih cepat dibandingkan mencari kombinasi optimal secara menyeluruh [3].

Pseudocode rekomendasi produk:

```
Input: products, recentlyViewed, limit

viewedIds = produk yang sudah dilihat
categoryWeights = bobot kategori dari histori lihat

Untuk setiap product:
    jika product sudah dilihat atau stok <= 0:
        lewati
    categoryScore = bobot kategori product
    popularityScore =
        sold_units * 2 + rating_average * rating_count
    totalScore = categoryScore * 1000 + popularityScore

Urutkan kandidat berdasarkan totalScore terbesar
Ambil limit produk teratas
Output: daftar produk rekomendasi
```

Potongan kode implementasi:

```
return products
    .filter((product) =>
        !viewedIds.has(product.id) && product.stock > 0
    )
    .map((product) => {
        const categoryWeight =
            categoryWeights.get(product.category) ?? 0;
        const popularityScore =
            (product.sold_units ?? 0) * 2 +
            (product.rating_average ?? 0) *
            Math.min(product.rating_count ?? 0, 10);

        return { product,
            score: categoryWeight * 1000 + popularityScore };
    })
    .sort((a, b) => b.score - a.score)
    .slice(0, limit);
```

Contoh output analisis:

```
Input histori:
- Buyer melihat 3 produk kategori Fashion
- Limit rekomendasi = 4

Output yang diharapkan:
1. Produk Fashion dengan stok tersedia
2. Produk Fashion dengan sold_units/rating lebih tinggi
3. Produk yang belum pernah dilihat buyer
4. Produk stok 0 tidak muncul
```

Hasil analisisnya adalah Greedy cukup sesuai untuk rekomendasi awal PasarKita. Skor kategori membuat rekomendasi mengikuti minat terbaru buyer, sedangkan skor popularitas membantu produk yang lebih sering dibeli atau diberi rating tampil lebih tinggi. Pendekatan ini sejalan dengan gagasan sistem rekomendasi yang membantu pengguna memilih item dari banyak alternatif [2].

3.2 Analisis Greedy pada Sorting Produk

Sorting produk juga memakai pola Greedy karena setiap produk diberi skor lokal, lalu produk dengan skor terbaik ditampilkan lebih dulu. Jika user memilih sorting terlaris, skor utama adalah jumlah unit terjual. Jika user memilih rating tertinggi, skor utama adalah rating rata-rata.

Pseudocode sorting produk:

```
Input: allProducts, sortMode

Untuk setiap product:
    hitung sold_units
    hitung rating_average

Jika sortMode = sold_desc:
    urutkan berdasarkan sold_units terbesar

Jika sortMode = rating_desc:
    urutkan berdasarkan rating_average terbesar

Jika skor sama:
    produk terbaru tampil lebih dulu

Output: produk terurut
```

Contoh output analisis:

```
Mode sorting: Terlaris

Produk A - sold_units: 25
Produk B - sold_units: 12
Produk C - sold_units: 3

Output:
Produk A, Produk B, Produk C
```

Output tersebut menunjukkan bahwa sistem memilih produk terbaik berdasarkan skor lokal yang paling relevan dengan mode sorting. Tidak ada proses optimasi global karena kebutuhan fiturnya adalah ranking cepat.

3.3 Analisis Greedy pada Fee Marketplace

Fee marketplace adalah penerapan Greedy paling sederhana. Sistem mengambil subtotal, menghitung 2%, lalu langsung memakai hasilnya sebagai fee. Tidak ada percabangan strategi, diskon bertingkat, atau optimasi lanjutan.

Potongan kode fee:

```
const FEE_PERCENTAGE = 0.02;

const calculateFee = (subtotal) => {
  const fee = Math.round(subtotal * FEE_PERCENTAGE);
  const total = subtotal + fee;
  return { subtotal, fee_marketplace: fee, total };
};
```

Contoh output analisis:

```
Input:
subtotal = Rp50.000

Proses:
fee = round(50000 * 0.02)
fee = 1000
total = 50000 + 1000

Output:
subtotal          = Rp50.000
fee marketplace = Rp1.000
total             = Rp51.000
```

Kompleksitas proses ini adalah $O(1)$ karena jumlah operasi tetap sama berapa pun jumlah produk di sistem. Rumus yang dianalisis adalah:

$$fee = round(subtotal \times 0.02) \quad (1)$$

3.4 Analisis KMP pada Pencarian Produk

String Matching digunakan ketika user mengetik keyword pada fitur pencarian. Pada PasarKita, pencarian produk memakai KMP secara case-insensitive. KMP dipilih karena tidak selalu mengulang pencocokan dari awal ketika terjadi mismatch. Algoritma ini menggunakan prefix table untuk menentukan posisi pattern berikutnya [4].

Pseudocode KMP:

```

Input: text, pattern

Bangun prefixTable dari pattern
i = 0 // index text
j = 0 // index pattern

Selama i < panjang text:
    jika text[i] == pattern[j]:
        i++
        j++

    jika j == panjang pattern:
        return true

    jika mismatch:
        jika j != 0:
            j = prefixTable[j - 1]
        selain itu:
            i++

return false

```

Potongan kode implementasi:

```
function kmpSearch(text, pattern) {
  if (!pattern || pattern.length === 0) return true;
  if (!text || text.length === 0) return false;

  const t = text.toLowerCase();
  const p = pattern.toLowerCase();
  const prefixTable = buildPrefixTable(p);

  let i = 0, j = 0;
  while (i < t.length) {
    if (t[i] === p[j]) { i++; j++; }
    if (j === p.length) return true;

    if (i < t.length && t[i] !== p[j]) {
      j = j !== 0 ? prefixTable[j - 1] : 0;
      if (j === 0) i++;
    }
  }
  return false;
}
```

Contoh output analisis:

```
Input:
text    = "Sepatu Running Lokal"
pattern = "sep"

Output:
true

Alasan:
"sep" ditemukan pada awal kata "Sepatu"
dan pencarian tidak membedakan huruf besar/kecil.
```

Kompleksitas KMP adalah:

$$T(n, m) = O(n + m) \quad (2)$$

dengan n sebagai panjang teks dan m sebagai panjang pattern. Pattern matching sendiri merupakan masalah dasar dalam pemrosesan string [7], dan algoritma string matching banyak dipakai dalam sistem pencarian informasi [6].

3.5 Analisis String Matching pada Pencarian Pesanan

Pencarian pesanan memiliki kebutuhan yang sedikit berbeda dari pencarian produk. Seller tidak hanya mencari berdasarkan nama produk, tetapi juga dapat mencari ber-

dasarkan order ID, transaction ID, dan tracking ID. Karena itu, pencarian pesanan dianalisis sebagai String Matching multi-field.

Pseudocode pencarian pesanan:

```
Input: orders, keyword

Untuk setiap order:
    cocok jika keyword ditemukan pada:
        - order_id
        - product_name
        - transaction_id
        - tracking_id

Output: daftar order yang cocok
```

Contoh output analisis:

```
Keyword:
"TRX-102"

Data:
Order 1 - transaction_id = "TRX-102-ABC"
Order 2 - transaction_id = "TRX-555-ZZZ"

Output:
Order 1
```

Hasil analisisnya adalah pencarian multi-field membuat fitur seller lebih praktis. Seller tidak perlu mengingat field tertentu; cukup mengetik potongan ID transaksi, nama produk, atau nomor tracking.

3.6 Kesimpulan Analisis Output

Secara keseluruhan, algoritma yang dipakai PasarKita tidak dibuat terlalu kompleks. Greedy dipilih karena cepat dan cukup untuk keputusan ranking sederhana. KMP dipilih karena efisien untuk pencarian exact substring. Keterbatasannya, Greedy belum menjadi sistem rekomendasi personal yang kompleks dan KMP belum mendukung typo atau fuzzy search. Pengembangan berikutnya dapat mengarah ke Epsilon Greedy untuk rekomendasi adaptif [5] atau fuzzy search untuk pencarian yang lebih toleran terhadap salah ketik.

Tabel 5: Output dari Penerapan Algoritma

Algoritma	Fitur	Output yang Terlihat di Aplikasi
Greedy	Rekomendasi produk	Produk relevan tampil pada bagian rekomendasi
Greedy	Sorting produk	Produk terlaris/rating tertinggi tampil lebih atas
Greedy	Fee marketplace	Subtotal, fee 2%, dan total pembayaran tampil konsisten
KMP	Pencarian produk	Produk yang mengandung keyword muncul di hasil pencarian
String Matching	Pencarian pesanan	Order yang cocok dengan ID/nama produk/tracking tampil di daftar seller

DAFTAR PUSTAKA

- [1] T. Oliveira, M. F. Martins & U. N. D. Lisboa, “Literature Review of Information Technology Adoption Models at Firm Level,” vol. 14, no. 1, 110–121, 2011.
- [2] P. Resnick, H. R. Varian & G. Editors, “ecommmender Systems mmende tems,” vol. 40, no. 3, 56–58, 1997.
- [3] J. Tanujaya & D. Y. Musyaffa, “Applied Information Technology and Computer Science Perbandingan Kinerja Algoritma Greedy dan Dynamic Programming dalam Optimasi Diskon Keranjang Belanja E- Commerce Menggunakan Dataset Online Retail UCI,” vol. 5, no. 1, 220–229, 2026.
- [4] N. Marbun, A. Rozy, S. A. Pasaribu, E. Bu, N. N. Hasibuan & M. Riansyah, “Publisher : Faatuatua Media Karya Implementasi Algoritma Knuth-Morris-Pratt Pada E-Katalog Perpustakaan Abstrak Identifikasi Masalah Pengumpulan Data Implementasi Algoritma Knuth- Hasil Analisis,” vol. 02, no. 02, 2–5, 2024.
- [5] I. Umami & L. Rahmawati, “Comparing Epsilon Greedy and Thompson Sampling model for Multi-Armed Bandit algorithm on Marketing Dataset,” vol. 2, no. 2, 14–26, 2021.
- [6] A. Al-howaide, W. Mardini, Y. Khamayseh & M. B. Yasin, “Fast string matching algorithm,”
- [7] M. Crochemore & C. Hancart, “Pattern matching in strings Pattern Matching in Strings,” 2013.